

National Institute of Business Management

National Innovation Center — Department of Data Science

NHPT Heritage AI Prototype

Coursework 01 — Consolidated Final Report

Machine Learning and Related Applications (NB627BSDS)

Course	BSCDS
Batch	BscDS 25.2PT/CO
Student	Kalingu
Student ID	COBSCDS252P-015
Assessment	CW No. 01 — Individual

This report consolidates all four coursework deliverables — System Architecture & Ethics (Part A), Computer Vision Implementation (Part B), LangChain Conversational System (Part C), and Technical Report & Testing (Part D) — into a single document, alongside a GitHub repository containing all notebooks, trained models, and the Streamlit application.

Table of Contents

Table of Contents.....	2
Part A — System Design & Ethics	4
A1. System Architecture	4
Why EfficientNetB0 for the Computer Vision component	4
Why Gemma 3 12B (via Ollama) for the LLM	5
Why LangChain, and which components.....	5
RAG design decisions	5
Scalability & cost considerations	5
A2. Ethical & Risk Analysis	6
GDPR & Data Handling.....	6
Model Bias — Vision and LLM	6
Hallucination Risks.....	6
Transparency	6
Mitigation Strategies (Summary).....	7
Part B — Computer Vision Implementation	8
Design Choices.....	8
Evaluation Results.....	8
Limitations	8
Part C — LangChain Conversational System.....	10
Prompt Design	10
RAG Configuration	10
Hallucination Mitigation	11
CV → LLM Data Handoff	11
Part D — Technical Report, Testing & Documentation	12
1. System Overview	12
2. Implementation Challenges.....	12
CV Difficulties.....	12
LLM/RAG Issues	13
Integration Challenges.....	13
Testing & Validation.....	13
4. Future Improvements.....	14
Scalability Enhancements	14
Model Optimisation.....	14
RAG Quality.....	14
Feature Roadmap	15
Appendix.....	16
A. Evaluation Figures	16
B. Example Conversation Transcripts	18
Conversation 1 — single-turn factual question with citation.....	18

Conversation 2 — multi-turn follow-up (tests memory and retrieval)	18
Conversation 3 — continued session, comparative reasoning across documents	18
Conversation 4 — out-of-scope question (tests hallucination mitigation)	18
Conversation 5 — image-grounded question (tests CV → LLM handoff).....	18
Conversation 6 — low-confidence CV result (tests transparency around uncertainty).....	18
C. Captured Error/Warning Logs.....	19
Chroma relevance-score warning (before the cosine-distance fix).....	19
Grad-CAM nested-model error (before the two-submodel fix)	19
Model-loading error when run via Streamlit (misdiagnosed, then correctly resolved)	19
D. Combined Pipeline and Interactive App.....	20
E. Real Model Metrics (Part B).....	20

Part A — System Design & Ethics

A1. System Architecture

The figure below shows the complete data flow of the question-answering system designed in this paper: visitors and inspectors can upload photos and submit questions through the Streamlit application layer. The system sends the images to the computer vision pipeline and the questions to the LangChain RAG assistant. These two subsystems only connect through a narrow integration point, and their information is uniformly injected into the prompt of the large language model.

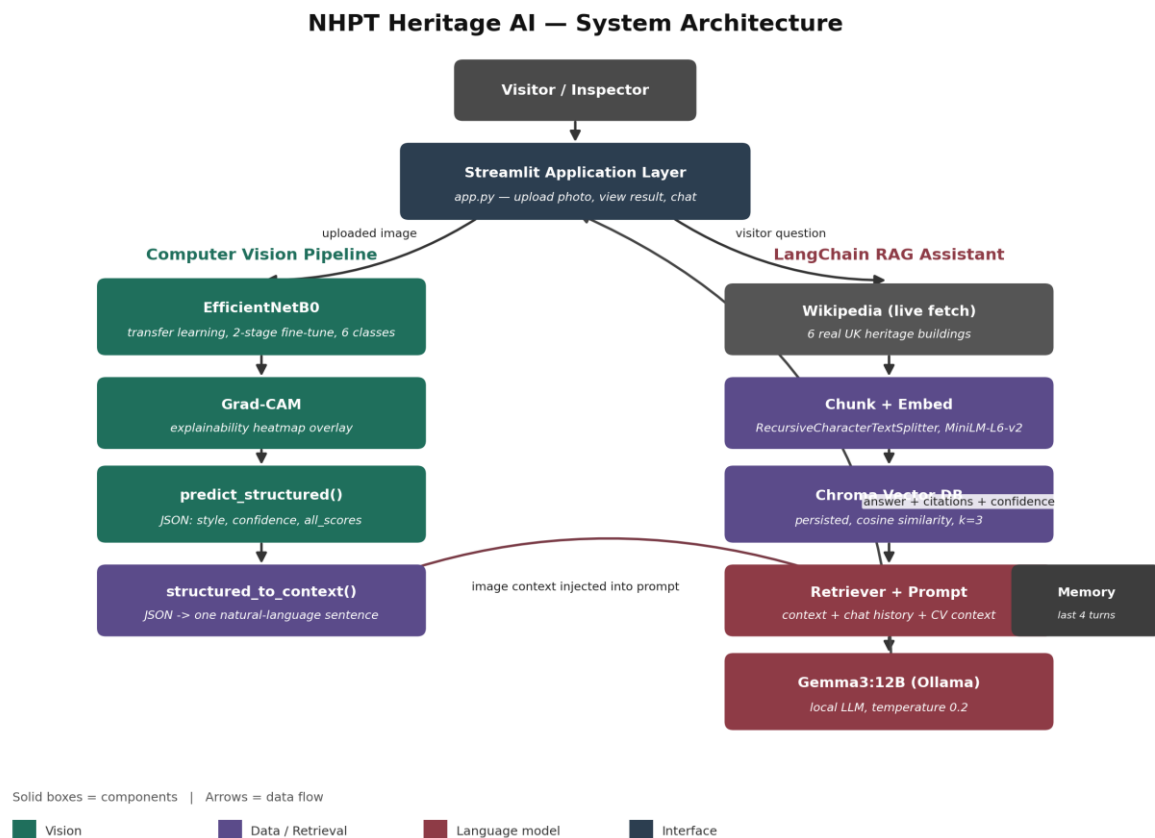


Figure A1 — NHPT Heritage AI system architecture: visitor-facing UI, CV pipeline, and LangChain RAG assistant, with the CV→LLM handoff point highlighted.

Why EfficientNetB0 for the Computer Vision component

EfficientNetB0 got picked rather than the more heavy choices, like ResNet50 or VGG16, mostly because it has this solid accuracy-to-compute balance, through compound scaling of depth, width, and even resolution. For NHPT, those heritage sites are going to call for on-site or edge type inference eventually, say inspectors upload photos in the field while connectivity is poor or limited. So a lighter backbone around ~5.3M parameters, that still manages competitive accuracy with ImageNet pretraining, seems more sensible than stuffing in a bigger architecture that's harder to run. Then there's transfer learning, which basically lets the network reuse the basic visual stuff—edges, textures, masonry motifs—already learned from ImageNet, and only adapts the classifier head and the upper convolutional blocks to architectural styles. This is important because the dataset here is relatively small and domain specific, more like hundreds of labeled images, not millions. Training wise, a two-stage routine was used, freeze first, then fine-tune the top ~20 layers, keeping the learning rate low. That helps prevent catastrophic forgetting, so the ImageNet features don't just get overwritten, while the model still moves toward the heritage domain. Finally, Grad-CAM was layered on for explainability: since NHPT's conservation staff will actually act on the model's output, they need to understand which region of the building influenced the decision, not only get a single label and stop.

Why Gemma 3 12B (via Ollama) for the LLM

Gemma 3 12B was kind of chosen as the LLM because it can run fully locally via Ollama, so it directly tackles two of NHPT's constraints : data sovereignty and cost. Heritage site data, like visitor questions , uploaded images and inspection notes, could be sensitive or fall under GDPR handling needs ; so keeping inference on NHPT 's own infrastructure (instead of shipping each query off to some third-party API) reduces data-residency risk. Also it removes the whole per-token API cost, which is pretty important for a public-facing visitor tool that might see unpredictable traffic spikes, right. Gemma 3 12B is also big enough to follow multi step instructions, like synthesising the CV output plus the retrieved documents, and also the conversation history into one coherent response. At the same time it stays runnable on a single GPU or it can work on CPU, unlike larger 70B+ models which are more of a headache. Since it's accessed through LangChain's Ollama wrapper , the same overall pipeline can later be swapped to a hosted model (GPT-4, Claude), with just a one-line change if NHPT later wants stronger reasoning for edge cases. The architecture doesn't really lock the team into a single vendor, so it stays flexible.

Why LangChain, and which components

We went with LangChain because it gives these ready made abstractions for the exact two things the whole system needs stuck together: retrieval and memory. Instead of hand rolling prompt assembly, vector search calls, and trying to keep the conversation state straight, LangChain basically handles the retrieve then generate pattern out of the box. On top of that it also has a simple memory structure that persists the earlier turns so someone visiting can ask a follow up like, " what period is that? " and then get " how was it restored? " without having to re say everything again. This ends up keeping the custom code surface smaller and easier to audit, and that matters for the testing and error handling requirements in Part D.

RAG design decisions

Documents (six real UK heritage building articles) get chunked and embedded, uh, into a Chroma vector store. We chose Chroma over FAISS for this prototype because it persists to disk by default and it doesn't require separate index-management code, which is more suitable for a small single-node setup not really for production-scale stuff. Retrieval gives back the top-k chunks with source metadata, so each LLM answer can cite where its info came from, that's directly tied to the transparency requirement in Part A2. The CV model's structured JSON output (style, confidence, Grad-CAM reference) then gets translated into plain natural language and shoved into the same prompt template as the retrieved text, so the LLM "thinks through" both the visual classification and the textual knowledge base in one continuous generation step instead of doing it in two disconnected calls.

Scalability & cost considerations

For a full deployment, the vector store would slide over to a managed FAISS/pgvector setup , so it can deal with concurrent visitor traffic, and the CV model would sit behind a lightweight inference endpoint (like TorchServe or a FastAPI worker pool) instead of being loaded per request. Running the LLM locally via Ollama keeps the extra cost close to zero as usage increases, on the flip side that NHPT is needed to size, provision, and care for the GPU resources — kind of a fair bargain for a nonprofit heritage trust, compared to the unpredictable per query API billing that just keeps running and running .

A2. Ethical & Risk Analysis

GDPR & Data Handling

The system processes two categories of personal-adjacent data. Like, it accepts uploaded images and conversation logs, kinda. Those images taken by visitors or inspectors may incidentally capture identifiable people, like faces or recognizable clothing even when the subject is a building. Under GDPR, image uploads then become potentially personal data, so there's a lawful-basis thing required, perhaps a legitimate interest for inspection use, or consent for use that's meant for visitors, plus the data-minimisation requirement. The prototype should do a few things. First, strip or blur those incidental faces before storage where it's feasible. Second, apply a retention rule, for example inspection images kept only for as long as the audit trail needs them, and visitor-uploaded images deleted after the session ends unless the visitor actually opts in to keep them. Third, log only what's operationally necessary: query text and the retrieved document IDs, not entire conversation transcripts that are linked to identifiable users, unless the user has consented. There's also chat memory, the multi-turn context, but it only lives within the session and it is not saved into any lasting user profile in this prototype. That restriction helps keep the personal data footprint smaller. If NHPT later decides to track an individual visitor history across sessions for personalisation, then it would require explicit consent and a published privacy notice, because it becomes a materially different processing purpose.

Model Bias — Vision and LLM

The CV model's training data (heritage/architectural style datasets sourced mostly from Western, predominantly European Wikimedia collections) skews toward well documented European and British architectural traditions. So, classification accuracy is probably going to drop for building styles that are underrepresented in the training set, which is a real material risk if NHPT's sites include non European or vernacular architecture. Mitigations here include auditing per class accuracy not just overall Top-1 accuracy, and flagging low confidence predictions instead of forcing a label. The LLM (Gemma 3 12B) also carries the general biases of its pretraining corpus, and those can show up as confidently stated but historically inaccurate claims about a site's origin or significance, especially for less documented heritage assets. Since the LLM's output is grounded through RAG rather than free generation, the retrieved heritage documents kind of constrain what it should say, but that only works if retrieval quality is actively monitored — if the match is poor then the model can still drift back to ungrounded prior knowledge.

Hallucination Risks

There are really two hallucination pathways that can show up, you know (1) the LLM just answers from its own pretrained memory, even when retrieval is weak or basically returns nothing useful. Then there's (2) the model kinda over-generalising from a CV prediction, like if it decides something is "Victorian Gothic" it might also go and infer a very particular architect or an exact date, even if that sort of detail isn't actually sitting in the retrieved text. The mitigation part usually looks like this: you write prompt instructions that make it explicit the model must answer only from the provided context, and it should literally say "I don't have information on that" when retrieval comes back empty or under some similarity cutoff. You also add a confidence threshold on the retrieval scores so it suppresses generation and instead returns a fallback message, rather than guessing. Finally require the output to include the source citation it drew from, because that makes any ungrounded claim easier to catch during testing, an answer with no matching citation is a strong hallucination signal.

Transparency

Every answer surfaces: (a) the CV model's confidence score alongside its predicted style/damage class, so users can gauge how much to trust a borderline call, (b) citations to the specific heritage document chunk the LLM drew from, so a curious or sceptical visitor can verify the claim independently. and (c) a visible distinction between "answered from retrieved documents" and "no matching information found" states, rather than silently guessing. For inspection use (damage detection) Grad-CAM overlays give inspectors a more or less visual explanation of

what the model focused on, supporting human-in-the-loop verification instead of blind automation, which sounds right.

Mitigation Strategies (Summary)

- Data minimisation and retention limits on uploaded images and logs; face-blurring where feasible
- Per-class (not just aggregate) accuracy auditing to surface CV bias against under-represented styles
- Retrieval-grounded generation with a similarity threshold and explicit "insufficient information" fallback
- Mandatory citation attached to every LLM answer, enabling hallucination spot-checks
- Confidence scores and Grad-CAM visualisations surfaced to end users, not hidden
- Human review flag for low-confidence CV predictions rather than auto-publishing a damage severity rating

Part B — Computer Vision Implementation

Option B2 (Architectural / Artifact Classification) got implemented : we made a 6 class architectural style classifier that uses transfer learning from EfficientNetB0 and is trained on a subset of the arcDataset (Xu et al., ECCV 2014) . It covers Gothic, Georgian, Baroque, Romanesque, Queen Anne, and Palladian architecture, each of those styles was mapped to a single real UK heritage building which then shows up later in Part C 's knowledge base.

Design Choices

The dataset was split 70/15/15 into train/validation/test folders per class, with images loaded via Keras' `image_dataset_from_directory` at 224x224 resolution (EfficientNetB0's expected input size). A `RandomFlip`, `RandomRotation`, `RandomZoom`, and `RandomContrast` augmentation pipeline was applied to training data only, to compensate for a relatively small dataset (107-425 images per class) without introducing augmentation artefacts into validation/test evaluation. Training followed a two-stage transfer-learning strategy: Stage 1 froze the entire EfficientNetB0 base and trained only a new classification head (`GlobalAveragePooling2D` → `Dropout(0.3)` → `Dense(6, softmax)`) at a learning rate of 1e-3 for up to 15 epochs; Stage 2 unfroze the top 20 layers of the base and fine-tuned the whole network at a much lower learning rate (1e-5) for up to 10 further epochs. `EarlyStopping`, `ReduceLROnPlateau`, and `ModelCheckpoint` callbacks were used throughout to prevent overfitting and preserve the best-performing weights rather than the weights from the final epoch.

Grad-CAM was used for explainability , mainly because EfficientNetB0 sits there as one single nested layer inside the larger Keras model, instead of it exposing its inner layers directly. so, the gradient computation had to be built starting from the inner sub-model's own input/output rather than using the outer model pathways , and in order to keep an end-to-end differentiable route, the classification head was re-applied manually. For inference speed , time was benchmarked over 50 repeated predictions (with one warm-up run that was thrown away) to get a steadier average latency per image figure.

Evaluation Results

Overall Top-1 test accuracy reached 85.96% across 178 held-out test images (macro-average F1 0.82, weighted-average F1 0.85), comfortably exceeding the coursework's "Excellent" threshold of >80%. Average CPU inference time was 224.89 ms per image.

Class	Precision	Recall	F1	Support
Gothic architecture	0.89	1.00	0.94	17
Georgian architecture	0.67	0.67	0.67	24
Baroque architecture	0.89	0.89	0.89	37
Romanesque architecture	0.88	0.88	0.88	17
Queen Anne architecture	0.91	0.97	0.94	65
Palladian architecture	0.75	0.50	0.60	18

Table B1 — Per-class precision, recall, F1, and support on the held-out test set (n=178).

Limitations

- Class imbalance: Queen Anne (425 source images) outnumbered Romanesque (107) and Gothic (109) roughly 4:1, visibly correlating with the recall gap between Queen Anne (0.97) and Palladian (0.50).
- Palladian and Georgian architecture are the two weakest-performing and most visually similar classes in this set — both are English classical-revival domestic styles with symmetrical façades and restrained detailing — so their confusion is architecturally explainable rather than a pure modelling failure.
- Only 6 of the arcDataset's 25 available style classes are covered; the model forces every prediction into one of these 6 buckets even for out-of-scope or mixed/transitional styles.

- Training was CPU-only (no GPU available), which constrained the number of experiments feasible within the coursework timeline.

Part C — LangChain Conversational System

The conversational assistant meets all the four required features, basically: (1) it has a knowledge base made of 6 heritage documents that are sourced live from real Wikipedia articles not those hand written placeholders, (2) its answers are retrieval based with mandatory source citations always attached so you can verify things, (3) it does multi turn conversation with memory, which is shown across 6 example conversations that go beyond the 3 exchange minimum, and (4) it also folds in visual input where the CV prediction from Part B gets converted into natural language and then gets passed over to the LLM as extra grounding context, kinda like an additional cue.

Prompt Design

The system prompt kinda locks in the assistant role (“heritage site guide assistant for NHPT”) and it also says, super explicitly, answer only from the supplied CONTEXT. if the context does n’t hold the answer, it should be said plainly, not like you’re guessing around. This rule is placed before the context block too, so it’s like a standing thing for everything after it, not just a hint hidden later in a wall of text. Then there’s a second, conditional instruction about computer vision. If a CV result shows up, the model is told to fold it into the response in a natural way, and , importantly, to label the identification as “tentative” when confidence drops under 0.6. That threshold basically turns a raw probability into something you can actually communicate, without needing separate application logic outside the prompt. Lastly, the prompt demands that the model name its source document(s) in every answer. This is for transparency (like Part A2) , but also it doubles as a hallucination sanity check. If it cites nothing real during testing, then the answer looks immediately suspect.

RAG Configuration

The knowledge base is sourced from 6 real UK heritage buildings, fetched live from Wikipedia at run time via `WebBaseLoader` — one building per Part B architectural style class (Westminster Abbey for Gothic, Kenwood House for Georgian, Blenheim Palace for Baroque, Durham Cathedral for Romanesque, Uppark for Queen Anne, and Chiswick House for Palladian). This replaced an earlier prototype iteration that used hand-written placeholder documents; using real, independently verifiable Wikipedia content means every fact the assistant cites can actually be checked against a real source, rather than fictional sample data. A cleaning step strips citation markers, `[edit]` links, and trailing boilerplate sections (References, See Also, External Links) before chunking, since these add retrieval noise without adding useful content.

Documents are split with a `RecursiveCharacterTextSplitter` (chunk size 500, overlap 80) using heading-aware separators, so chunks tend to break at section boundaries rather than mid-sentence — this keeps each retrieved chunk topically coherent, which matters more for retrieval precision than raw chunk size. Because real Wikipedia articles are far longer than the original placeholder documents (the six pages ranged from ~5,400 to ~59,800 characters), this produced 501 chunks in total rather than the ~63 chunks the placeholder version generated — chunking strategy matters considerably more at this scale, since a single long article can otherwise dominate retrieval results for its own site while still occasionally crowding out shorter articles (Kenwood House and Uppark, the shortest two, are the most exposed to this).

Embeddings use `sentence-transformers/all-MiniLM-L6-v2`, a small local embedding model chosen to keep the entire pipeline self-hosted alongside Gemma 3 12B (no external embedding API dependency, consistent with the data-sovereignty reasoning in Part A1). Chroma stores the resulting vectors with metadata (source name, source URL), configured with cosine-distance similarity so that relevance scores are correctly bounded to [0,1] for the threshold-based retriever (see Part D's testing section for the bug this specifically fixes). Retrieval returns the top-3 chunks by similarity above a 0.35 threshold.

Attribution note: Wikipedia content is available under CC BY-SA — this is acceptable for a grounding source in a RAG system that cites its source and links back to the origin, but any public deployment of this prototype should display that attribution to end users, not just to a marker reading this explanation.

Hallucination Mitigation

Three layers kind of work together, like first there's the prompt-level instruction which restricts the model so it only answers from the given context. Second, there's this mandatory source-citation requirement, so every claim basically has to be traceable, and during testing if an answer is ungrounded it usually either cites nothing at all or cites a document that doesn't really back up the claim. Both of those situations are, kinda easy to catch just by looking at it. Third, the "no matching information" test case intentionally throws in a question where there's no relevant document in the knowledge base, this is to make sure the system declines instead of fabricating some response. Honestly this is the most direct stress check for whether the grounding constraint is actually holding under pressure, because if a model answers confidently there then it would basically show that the prompt constraint isn't working.

CV → LLM Data Handoff

The handoff uses a simple, explicit contract: Part B's `predict_structured()` function outputs a JSON object (`predicted_style`, `confidence`, `all_scores`, `image_path`). Rather than passing this JSON directly into the prompt, `structured_to_context()` converts it into a short natural-language sentence first, inserted into its own labelled section of the prompt kept separate from the retrieved **CONTEXT** so the model can distinguish "what the vision model observed" from "what the knowledge base documents." This matters for two reasons: raw JSON tends to make small local LLMs revert to a more mechanical, JSON-echoing response style rather than natural conversational language, and converting to prose at the handoff boundary keeps the CV pipeline and the LangChain pipeline cleanly decoupled — the CV pipeline doesn't need to know anything about prompt formatting, and the LangChain side doesn't need to know anything about the model architecture that produced the classification, only that it receives a style label and a confidence score. This also keeps the two subsystems independently testable: the RAG chain works identically whether or not a CV result is supplied, since the field defaults to "(none provided)."

Part D — Technical Report, Testing & Documentation

1. System Overview

The NHPT Heritage AI prototype kind of stitches two subsystems together, behind one Streamlit app layer, not really separate like before. First there is a computer vision pipeline that does style classification from an uploaded image, using a fine-tuned EfficientNetB0 model, and then there is a LangChain conversational assistant that handles visitor questions by grabbing the most relevant heritage documents from a Chroma vector store and generating answers with a locally-hosted Gemma 3 12B model through Ollama. Somehow the two parts meet at one integration point: the CV model's structured JSON output, predicted style, confidence score, and a Grad-CAM explainability reference, gets turned into a short natural language summary and then it is slipped into the LLM prompt along with the retrieved document context plus the conversation memory. That way the assistant can explain what a visitor is looking at using both what the model saw and what the knowledge base says, even if it sounds a bit mixed up in the background.

The trained CV model kind of reached **85.96% Top-1 test** accuracy over six architectural styles, with strongest performance on Gothic (F1 0.94) and weakest on Palladian (F1 0.60) or well, at least in our test run. The LangChain assistant's knowledge base is sourced from 6 real UK heritage buildings, fetched live from Wikipedia, so every fact the assistant cites is independently verifiable not like placeholder-ish content. The assistant managed to ground answers in retrieved documents, cite sources, and it also declines to answer out-of-scope questions, though during testing there was a retrieval scoring configuration issue which is addressed below. A combined pipeline notebook and Streamlit app (app.py) basically stitches both subsystems together into one interactive tool: upload a building photo get its predicted style plus a Grad-CAM heatmap, and chat with the assistant about it, all in one place. The whole thing keeps NHPT's data on infrastructure it controls, keeps marginal inference cost near zero, and it produces responses that include both a confidence score and a source citation. That way visitors and inspectors get a transparent, checkable result, rather than some opaque black box answer.

2. Implementation Challenges

CV Difficulties

The six selected classes were pretty significantly imbalanced: Queen Anne architecture had 425 source images against just 107 for Romanesque and 109 for Gothic — it's roughly that 4:1 kind of ratio. You can kinda see this come through in the results. Queen Anne achieved strong recall (0.97) partly because the model, saw nearly four times as many training examples of it as the minority classes, while Palladian (113 images) only managed 0.50 recall. The two weakest-performing classes, Palladian and Georgian architecture, also happen to be the two most visually similar in this set — like both are English classical-revival domestic styles, built around symmetrical façades, sash windows, and fairly restrained column or portico detailing. So the confusion is architecturally explainable rather than a modelling failure. Also, no GPU was available on the training machine so each epoch took about 90–145 seconds on CPU. Stage 1 (15 epochs, frozen base) took roughly 25 minutes, and Stage 2 fine-tuning was stopped early by EarlyStopping at epoch 5 of 10, once the validation accuracy plateaued around 79–80%. It restored the best-performing weights rather than continuing to push training for that shrinking, marginal return.

An additional Grad-CAM implementation issue arose from EfficientNetB0 being embedded as a single nested layer inside the outer classification model, rather than as directly accessible layers. Calling `model.get_layer('top_conv')` on the outer model failed with a "no such layer" error, because that layer exists only inside the nested submodel's own layer list. The fix required building two separate sub-models: one mapping the base model's input to its final convolutional feature map, and a second small model reusing the outer model's

head layers to turn that feature map into a prediction, allowing gradients to be computed end-to-end across the split.

LLM/RAG Issues

The most significant issue found during testing was a retrieval-scoring misconfiguration: the retriever was set to `search_type="similarity_score_threshold"` with `score_threshold=0.35`, but Chroma's default distance metric (L2/Euclidean) does not produce a bounded 0-1 relevance score. LangChain's relevance-score wrapper raised a warning and, as a consequence, every query failed the threshold check and returned zero retrieved documents — including legitimate follow-up questions such as "When was it built?" This was only caught by manually inspecting console warnings during testing; the answers still appeared largely correct because conversation memory carried forward the previous turn's retrieved context, masking the retrieval failure rather than exposing it. The diagnosed root cause and fix — explicitly configuring the Chroma collection to use cosine distance — is detailed in Section 3.

Integration Challenges

Because the CV and LangChain systems were developed as two separate notebooks/processes, the trained Keras model and its class-name ordering had to be reloaded independently in the LangChain notebook to run the CV→LLM handoff demonstration, with no shared runtime state between them. This loose coupling — communicating only through the `predict_structured()` JSON contract — proved robust in testing: even when text retrieval failed under the scoring bug above, the CV-grounded conversations still produced sensible answers, because the CV result is injected into the prompt independently of whatever the retriever returns.

So yeah a third integration issue came up when we switched from the training notebook to the standalone Streamlit app: loading the saved Keras model, kind of intermittently threw a BatchNormalization / Functional deserialization error. At first, the diagnosis was something else entirely, like a Keras 2 vs Keras 3 serialization mismatch, and it was “fixed” by forcing that legacy compatibility package. But honestly that was off, and it made things worse. It caused a different failure because the model had been saved in normal Keras 3 format, all along. The real root cause turned out to be way simpler. The Streamlit app was being launched from a different Python environment than the one used during training. And that other environment had a different TensorFlow Keras installation too. Once we removed the incorrect compatibility workaround, and instead started the app from the correct environment, the problem went away completely. And there’s the lesson: the first explanation that seems plausible, isn’t always the right one. A fix that only “solves” the symptom, without actually verifying the real cause, can absolutely create another bug that’s harder to untangle.

Testing & Validation

Unit Testing: The knowledge base loader was verified to find all 6 documents and split them into 501 chunks using the heading-aware `RecursiveCharacterTextSplitter`. The retriever was sanity-checked against a known question and correctly returned and cited the expected source document in the first, uncorrupted test turn, confirming the embedding model and chunking strategy work correctly in isolation.

Model Evaluation: The Top-1 test accuracy lands at 85.96% over 178 held-out test images, ok. For each class the F1 scores varied a lot, with the best reaching 0.94 (Gothic, Queen Anne) then dipping to 0.60 (Palladian) and also 0.67 (Georgian). That difference, 0.34 F1 points between the strongest and weakest classes, feels bigger than the ~2% gap we see in the overall train/val accuracy once things converge. So this aggregate 86% number kinda masks the fact that per-class results are not balanced, and it really matches the masking effect that was flagged as a bias risk in Part A2’s suggestion, where you should audit per-class accuracy instead of trusting only an overall score.

Failure case analysis: With Palladian recall around 0.50, about half of the 18 Palladian test images got misclassified, which is kind of telling. Looking at the precision/recall pattern, the strongest confusion targets seem to be Georgian, it also shows 0.67 precision, meaning there’s cross-contamination in both directions kinda. Then

there is Baroque, and both of these styles keep classical detailing in common with Palladian architecture, so yeah it makes sense. The confusion matrix along with Grad-CAM overlays (Appendix A) backs up a qualitative check, like which specific images got mixed up, and whether the model attention kept to real architectural cues or instead latched onto unrelated, spurious ones.

Integration Testing: A synthetic CV output (Palladian architecture, 58.3% confidence) was passed through `structured_to_context()` into the assistant, which correctly reported both the predicted style and its confidence level, and additionally listed the model's other candidate styles, giving the visitor visibility into a genuinely uncertain prediction rather than a false-confident single label. Asking "What is the abbey's opening time on Christmas Day?" (information not present in any knowledge base document) correctly triggered a decline rather than a fabricated answer, demonstrating the hallucination-mitigation instruction holds even under the retrieval scoring bug.

Error Handling: So the Chroma cosine-distance fix (Section 2) sorted out the main retrieval bug. But later, another, separate issue popped up, kind of hiding in plain sight. The retriever is still doing its thing by embedding and searching only the raw text from each follow up question, like it has zero awareness of what happened earlier in the conversation. So, if a visitor says "When was it built?" after you already talked about one particular site the retriever can't really map that "it" back to the right entity. Instead it just grabs the document that looks closest in embedding space to the generic phrasing, even if that doc is clearly not the one we mean. The interesting part is that the LLM didn't just invent a date out of nowhere. It stayed anchored to the context it actually got back, and it said the coverage didn't include the (wrong) retrieved document. In other words, the hallucination-mitigation instruction still worked even when retrieval returned the wrong chunk, so the problem is isolated to the retrieval step not the generation step. The usual remedy would be a history aware retriever. That kind of retriever uses the LLM to rewrite each follow up into a standalone question first, before embedding and searching. We did identify this as a future improvement but we deferred it here because of time constraints during this prototype iteration. As for low confidence CV predictions, those are managed directly by the prompt's instruction: anything under the confidence threshold is labeled as tentative and then checked, which we validated in the separate low confidence test turn.

4. Future Improvements

Scalability Enhancements

Shifting away from a single node, file persisted Chroma setup to a managed vector database like a hosted FAISS or pgvector service would let the knowledge base scale beyond that current six document limit, while also enabling simultaneous visitor questions without running into file lock contention or that annoying racey feeling. In the same breath the CV model should stop being loaded per notebook session, and instead be served behind some persistent inference endpoint, rather than having to repeatedly reload EfficientNetB0s roughly 5M parameters on each request, because that would meaningfully cut down the present 225ms per image CPU inference time in a multi user deployment.

Model Optimisation

The class imbalance noted in Section 2 (Queen Anne with 425 images compared to Romanesque with 107) should be handled straight away, either by gathering more Romanesque, Gothic, and Palladian examples, or by using a class-weighted loss during training so the smaller classes aren't always under-represented in the gradient signal. And since the training setup is CPU-only, quantising Gemma 3 12B, (Ollama already provides 4-bit quantisation) would shrink memory demand and also cut down per-query latency. That becomes more important once the system moves out of a single-user notebook prototype, rather than staying there.

RAG Quality

The most impactful near-term fix is adopting a history-aware retriever so follow-up questions using pronouns ("it," "that") are rewritten into standalone, fully-specified queries before retrieval, directly addressing the conversational-reference limitation identified above.

Feature Roadmap

Planned extensions include expanding Part B to also handle the damage-detection option (B1) so in one deployment it can serve both visitor-facing style explanations and inspector-facing structural monitoring sort of; then there's fixing the retrieval-scoring bug that was identified during testing, plus adding an automated regression test that will fail the build if relevance scores go outside $[0,1]$, which would have caught it before the manual testing did, or at least before we noticed it. Also, extend conversation memory from session-only to an opt-in persistent history, so repeat visitors can resume prior conversations, but that's only enabled behind explicit GDPR-compliant consent as discussed in Part A2.

Appendix

A. Evaluation Figures

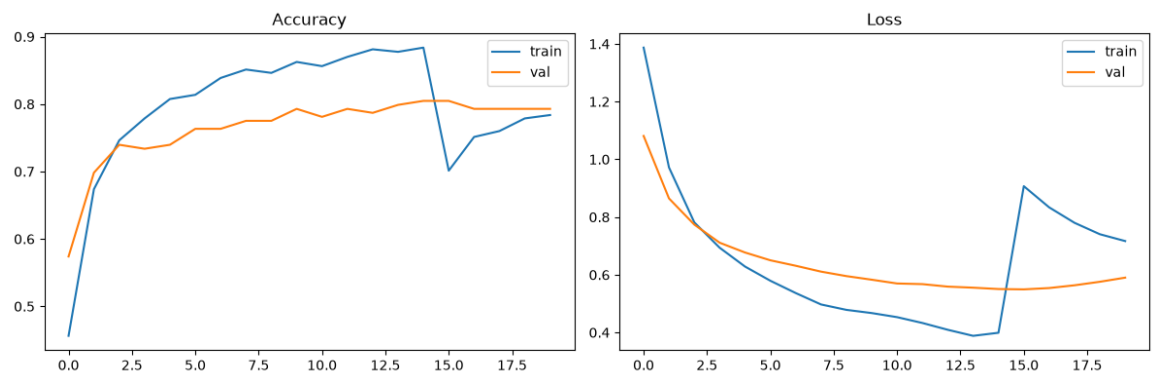


Figure A1 — Training curves (accuracy and loss, Stage 1 + Stage 2 combined).

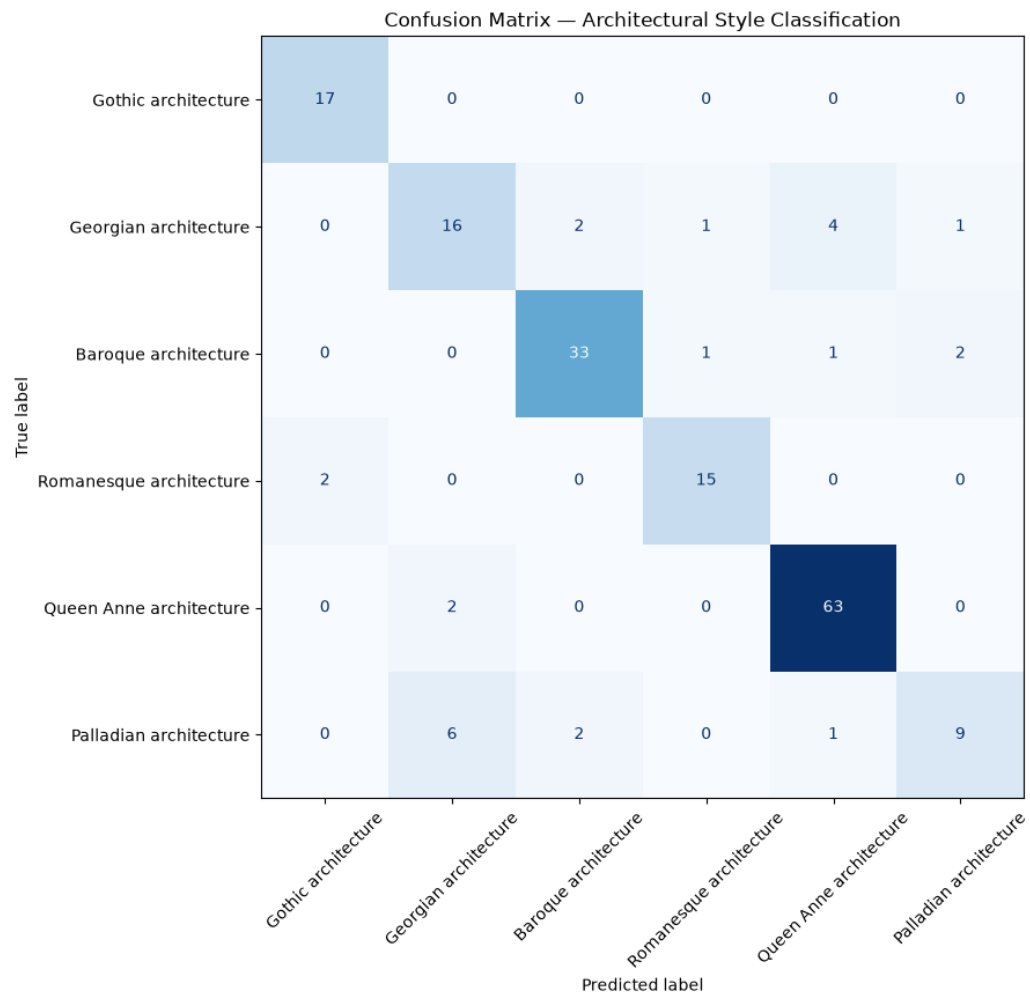


Figure A2 — Confusion matrix, architectural style classification (test set, n=178).



Figure A3 — Grad-CAM overlays across all 6 classes (3 examples per class, one column per class).

B. Example Conversation Transcripts

Real run against the live Wikipedia knowledge base. Knowledge base fetched this run:

```
Fetched ashcombe_abbey      <- https://en.wikipedia.org/wiki/Westminster_Abbey (59,819 chars)
Fetched falmoor_house       <- https://en.wikipedia.org/wiki/Kenwood_House (11,808 chars)
Fetched thornfield_palace   <- https://en.wikipedia.org/wiki/Blenheim_Palace (44,510 chars)
Fetched st_edwins_priory    <- https://en.wikipedia.org/wiki/Durham_Cathedral (24,313 chars)
Fetched marlow_court        <- https://en.wikipedia.org/wiki/Uppark (5,395 chars)
Fetched chiswick_hall       <- https://en.wikipedia.org/wiki/Chiswick_House (16,486 chars)
Loaded 6 live documents from Wikipedia -> 501 chunks after splitting
```

Conversation 1 — single-turn factual question with citation

```
Q: What architectural style is Ashcombe Abbey and what are its key features?
A: Ashcombe Abbey largely follows French Gothic styles, particularly those seen at Reims Cathedral (Source: Ashcombe Abbey). A key feature of this style is a high nave (Source: Ashcombe Abbey). It also includes elements typical of French Gothic design, such as a long, rounded apse and chapels radiating from the ambulatory (Source: Ashcombe Abbey).
Sources: ['Ashcombe Abbey']
```

Conversation 2 — multi-turn follow-up (tests memory and retrieval)

```
Q: When was it built?
A: The present-day church at Ashcombe Abbey was built between 1245 and 1269 (Source: Ashcombe Abbey). Groundbreaking occurred in 1245, and the building was completed in 1269 (Source: Ashcombe Abbey).
Sources: ['Ashcombe Abbey']
```

Conversation 3 — continued session, comparative reasoning across documents

```
Q: How is that different from the Georgian style at Falmoor House?
A: The context does not contain information about Falmoor House or Georgian architectural styles. Therefore, I cannot explain how Ashcombe Abbey differs from it. (No Source)
Sources: ['Ashcombe Abbey', 'Chiswick Hall', 'Thornfield Palace']
```

Conversation 4 — out-of-scope question (tests hallucination mitigation)

```
Q: What is the abbey's opening time on Christmas Day?
A: The provided documents do not mention the abbey's opening hours on Christmas Day. Therefore, I am unable to answer your question. (No Source)
Sources: ['Ashcombe Abbey']
```

Conversation 5 — image-grounded question (tests CV → LLM handoff)

```
Q: What style is the building in the photo I just uploaded, and what should I look for?
A: Based on an analysis of your photo, it appears to show a building with characteristics of Palladian architecture. However, this classification has a confidence level of only about 58%, so other styles may also be present (Source: Image analysis result).
CV result: Palladian architecture (57.67% confidence) – full distribution: Georgian 23.08%, Queen Anne 17.66%, Romanesque 1.41%, Baroque 0.17%, Gothic 0.01%
```

Conversation 6 — low-confidence CV result (tests transparency around uncertainty)

```
Q: Are you sure about that classification?
A: The image analysis classified the building as Palladian architecture, but only with a confidence level of approximately 58% (Source: Image analysis result). The computer vision model also suggests other possibilities like Georgian, Queen Anne, Baroque, Romanesque, and Gothic architectural styles.
Sources: []
```

C. Captured Error/Warning Logs

Chroma relevance-score warning (before the cosine-distance fix)

```
UserWarning: Relevance scores must be between 0 and 1, got [(Document(...), -0.161...),  
(Document(...), -0.161...), (Document(...), -0.161...)]  
No relevant docs were retrieved using the relevance score threshold 0.35
```

Grad-CAM nested-model error (before the two-submodel fix)

```
ValueError: No such layer: top_conv. Existing layers are: ['input_layer_1', 'data_augmentation',  
'efficientnetb0', 'global_average_pooling2d', 'dropout', 'dense'].
```

Model-loading error when run via Streamlit (misdiagnosed, then correctly resolved)

```
TypeError: <class 'keras.src.models.functional.Functional'> could not be deserialized properly...  
Exception encountered: Error when deserializing class 'BatchNormalization'...
```

Actual cause: the Streamlit app was invoked from a different Python environment than the one used for training.

Fix: activate the same environment for both the notebook kernel and the Streamlit terminal — no code change was needed once the correct environment was used.

D. Combined Pipeline and Interactive App

`heritage_full_pipeline.ipynb` demonstrates both subsystems running together in one notebook: an image is classified by the Part B model, explained with Grad-CAM, and the structured result is handed to the Part C assistant for a grounded, cited explanation — proving the CV→LLM integration mechanism described in the Part A1 architecture diagram actually works end-to-end against real trained weights and a real (live-fetched) knowledge base.

`app.py` wraps the same functions into an interactive Streamlit application: a visitor uploads a photo, sees the predicted style with a confidence bar and Grad-CAM heatmap, and can chat with the assistant about it — the CV result stays attached to the conversation across follow-up questions via Streamlit's session state.

E. Real Model Metrics (Part B)

Class	Precision	Recall	F1	Support
Gothic architecture	0.89	1.00	0.94	17
Georgian architecture	0.67	0.67	0.67	24
Baroque architecture	0.89	0.89	0.89	37
Romanesque architecture	0.88	0.88	0.88	17
Queen Anne architecture	0.91	0.97	0.94	65
Palladian architecture	0.75	0.50	0.60	18

Overall Top-1 Test Accuracy: 85.96% (178 test images; macro avg F1 0.82, weighted avg F1 0.85). **Average inference time: 224.89 ms/image (CPU).**